SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE	DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Program Name: B. Tech	**Assignment Type: Lab**	**Academic Year:**2025-2026
Course Coordinator Name	Dr. Rishabh Mittal	
Instructor(s)Name	Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpulasumalatha Mr. Bingi Raju	

CourseCode	23CS002PC304	**CourseTitle**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
DateandDay of Assignment	Week10 - Tuesday	**Time(s)**	
Duration	2 Hours	**Applicableto Batches**	23CSBTB01 To 23CSBTB52

Assignment Number: 19.2(Present assignment number)/**24**(Total number of assignments)

Q.No.	Question	*Expected Time to complete*
1	**Lab 19 – Code Translation: Converting Between Programming Languages** **Lab Objectives:** • Understand how AI tools can assist in translating code between different programming languages. • Learn to verify correctness and functionality after translation. • Explore syntactic and semantic differences between languages (e.g., Python, Java, C++). • Practice debugging and optimizing AI-translated code.	Week10 - Tuesday

Task 1 – Python to Java Conversion

Task:

Convert a Python program that performs file handling operations into an equivalent Java program using AI assistance.

Instructions:

- Prompt AI to translate a Python script that reads from and writes to a file into Java.
- Ensure proper handling of file streams, exceptions, and input/output formatting.
- Verify that the Java program produces the same file content and console output as the Python version.
- Test both implementations with sample input files.

Expected Output:

A correctly translated and executable Java program that performs the same file handling operations as the original Python script.

Task 2 – C++ to Python Function Conversion

Task:

Translate a C++ program that implements a sorting algorithm into Python using AI assistance.

Instructions:

- Provide AI with a C++ implementation of a sorting technique (e.g., Bubble Sort or Selection Sort).
- Ask AI to generate an equivalent Python implementation.
- Ensure logical correctness and validate results using multiple test cases.
- Confirm that the Python version maintains the same sorting behavior.

Expected Output:

A correct and executable Python program equivalent to the given C++ sorting implementation.

Task 3 – SQL to Pandas Query

Task:

Convert a SQL query involving GROUP BY and aggregate functions into an equivalent Pandas Data Frame operation.

Instructions:

- Provide AI with a SQL query that calculates aggregate values (e.g., total sales per department).
- Prompt AI to generate equivalent Pandas code using groupby() and aggregation functions.
- Test the generated code using a sample dataset.
- Verify that both SQL and Pandas outputs match.

Expected Output:

An accurate Pandas Data Frame operation that produces results equivalent to the given SQL query with aggregation.

Task 4 – Real-Time Application: Multi-Language Snippet Converter

Scenario:

Students will select a practical real-world application requiring the same logic in two different programming languages.

Example:

- Converting a REST API implementation from Python (Flask) to Java (Spring Boot).
- Translating a data processing script from Python to C++ for performance optimization.

Instructions:

- Provide AI with source code in one programming language.
- Prompt AI to convert the code into another chosen language.
- Execute and test both versions to ensure identical functionality.
- Document differences observed in syntax, libraries, and execution behavior.

Expected Output:

Functionally equivalent and executable code snippets implemented in two different programming languages, along with documented observations.

Task -5: (Basic Function Translation (Python to C++)

Translate a simple Python function that checks whether a number is even or odd into C++ using AI assistance.

Instructions:

- Provide AI with a basic Python function that determines if a number is even or odd.
- Prompt AI to generate equivalent C++ code.
- Ensure correct syntax, input handling, and output formatting.
- Compile and run the C++ program to verify it produces the same result as the Python version.
- Test with at least three different input values.

Expected Output:

A simple and executable C++ program equivalent to the given Python function, correctly identifying whether a number is even or odd for all test cases.

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

Task 1: Python to Java Conversion

Prompt:

Convert a Python program that performs file handling operations (reading and writing) into an equivalent Java program. Ensure proper file streams, exception handling, and identical input/output behavior.

Code:

Python Code (Original):

```python
# Python file handling program

# Write to file
with open("sample.txt", "w") as file:
    file.write("Hello World\n")
    file.write("AI Assisted Coding Lab")

# Read from file
with open("sample.txt", "r") as file:
    content = file.read()
    print("File Content:\n", content)
```

Java Code (Converted):

```java
import java.io.*;
public class Main {
    public static void main(String[] args) {
        try {
            // Write to file
            FileWriter writer = new FileWriter("sample.txt");
            writer.write("Hello World\n");
            writer.write("AI Assisted Coding Lab");
            writer.close();

            // Read from file
            FileReader reader = new FileReader("sample.txt");
            BufferedReader br = new BufferedReader(reader);

            String line;
            System.out.println("File Content:");
            while ((line = br.readLine()) != null) {
                System.out.println(line);
            }

            br.close();
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Sample Input/Output:

Python:

```
File Content:
 Hello World
AI Assisted Coding Lab
```

Java:

```
File Content:
Hello World
AI Assisted Coding Lab
```

Code Explanation:

1. This task demonstrates translation of file handling code from Python to Java.
2. The Python program uses open() with read and write modes.
3. It writes data to a file and then reads and prints the content.
4. The Java version uses FileWriter and BufferedReader for file operations.
5. Exception handling is implemented using try-catch blocks in Java.
6. Both programs perform identical operations and produce the same output.
7. The translation highlights differences in syntax and file handling mechanisms.
8. This ensures functional equivalence between both programming languages.

Task 2: C++ to Python Function Conversion

Prompt:

Convert a C++ program implementing a sorting algorithm (Bubble Sort) into an equivalent Python program. Ensure the Python version maintains the same logic and produces identical sorted output for multiple test cases.

Code:

C++ Code (Original – Bubble Sort):

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[] = {5, 2, 9, 1, 6};
    int n = 5;

    // Bubble Sort
    for(int i = 0; i < n-1; i++) {
        for(int j = 0; j < n-i-1; j++) {
            if(arr[j] > arr[j+1]) {
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }

    cout << "Sorted Array: ";
    for(int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}
```

Python Code (Converted):

```python
# Python Bubble Sort

arr = [5, 2, 9, 1, 6]
n = len(arr)

# Bubble Sort
for i in range(n-1):
    for j in range(n-i-1):
        if arr[j] > arr[j+1]:
            arr[j], arr[j+1] = arr[j+1], arr[j]

print("Sorted Array:", arr)
```

Sample Input/Output:

C++ Code (Original – Bubble Sort):

```
Sorted Array: 1 2 5 6 9
```

Python Code (Converted):

```
Sorted Array: [1, 2, 5, 6, 9]
```

Code Explanation:

1. This task demonstrates translation of a sorting algorithm from C++ to Python.
2. The C++ program uses Bubble Sort to arrange elements in ascending order.
3. Nested loops compare adjacent elements and swap them if needed.
4. The Python version maintains the same logic using for loops.
5. Python simplifies swapping using tuple unpacking.
6. Multiple test cases are used to verify correctness.
7. Both implementations produce identical sorted results.
8. This confirms logical equivalence between C++ and Python versions.

Task 3: SQL to Pandas Query

Prompt:

Convert a SQL query using GROUP BY and aggregate functions into an equivalent Pandas DataFrame operation.

Ensure the Pandas output matches the SQL query results using a sample dataset.

Code:

SQL Query (Original)

```sql
SELECT department, SUM(sales) AS total_sales
FROM sales_data
GROUP BY department;
```

Python Code (Pandas Equivalent)

```python
import pandas as pd

# Create dataset
data = pd.DataFrame({
    "department": ["CSE", "ECE", "CSE", "ECE", "MECH"],
    "sales": [100, 150, 200, 100, 250]
})

print("Original Data:")
print(data)

# Group by and aggregate
result = data.groupby("department")["sales"].sum().reset_index()

print("\nTotal Sales per Department:")
print(result)
```

Sample Input/Output:

SQL Query:

```
| department | sales |
|------------|-------|
| CSE        | 100   |
| ECE        | 150   |
| CSE        | 200   |
| ECE        | 100   |
| MECH       | 250   |
```

Python Code (Pandas Equivalent)

```
Original Data:
  department  sales
0        CSE    100
1        ECE    150
2        CSE    200
3        ECE    100
4       MECH    250

Total Sales per Department:
  department  sales
0        CSE    300
1        ECE    250
2       MECH    250
```

Code Explanation:

1. This task demonstrates converting SQL aggregation queries into Pandas operations.
2. The SQL query groups data by department and calculates total sales.
3. The Pandas version uses groupby() to achieve the same functionality.
4. The sum() function aggregates sales values for each department.
5. The dataset is created manually to simulate database records.
6. The result is reset using reset_index() for proper table format.
7. Both SQL and Pandas outputs produce identical results.
8. This confirms correct translation between SQL and Python data processing.

Task 4: Real-Time Application: Multi-Language Snippet Converter

Prompt:

Convert a Python program that processes a list of numbers (finding sum and average) into an equivalent C++ program. Ensure both implementations produce identical outputs and analyze differences in syntax and execution.

Code:

Python Code (Source)

```python
# Python program for data processing

numbers = [10, 20, 30, 40, 50]

total = sum(numbers)
average = total / len(numbers)

print("Sum:", total)
print("Average:", average)
```

C++ Code (Converted)

```cpp
#include <iostream>
using namespace std;

int main() {
    int numbers[] = {10, 20, 30, 40, 50};
    int n = 5;

    int total = 0;

    for(int i = 0; i < n; i++) {
        total += numbers[i];
    }

    float average = (float) total / n;

    cout << "Sum: " << total << endl;
    cout << "Average: " << average << endl;

    return 0;
}
```

Sample Input/Output:

Python Code (Source)

```
Sum: 150
Average: 30.0
```

C++ Code (Converted)

```
Sum: 150
Average: 30
```

Code Explanation:

1. This task demonstrates converting a real-world data processing program between languages.
2. The Python program calculates sum and average using built-in functions.
3. The C++ version uses loops to perform the same calculations manually.
4. Both implementations process the same dataset and produce identical results.
5. Python offers simpler syntax and built-in utilities like sum().
6. C++ requires explicit loops and type casting for calculations.
7. Execution behavior differs as Python is interpreted and C++ is compiled.
8. This highlights differences in syntax, libraries, and performance.

Task 5: (Basic Function Translation (Python to C++)

Prompt:

Convert a Python function that checks whether a number is even or odd into an equivalent C++ program. Ensure proper input handling, syntax correctness, and identical output behavior for multiple test cases.

Code:

Python Code (Original)

```python
# Python function to check even or odd

def check_even_odd(num):
    if num % 2 == 0:
        print("Even")
    else:
        print("Odd")

# Input
num = int(input("Enter a number: "))
check_even_odd(num)
```

C++ Code (Converted)

```cpp
#include <iostream>
using namespace std;

void checkEvenOdd(int num) {
    if (num % 2 == 0)
        cout << "Even";
    else
        cout << "Odd";
}

int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;

    checkEvenOdd(num);

    return 0;
}
```

Sample Input/Output:

Python Code (Original)

```
Enter a number: 45
Odd
```

C++ Code (Converted)

```
Enter a number: 45
Odd
```

Code Explanation:

1. This task demonstrates basic function translation from Python to C++.
2. The Python function checks whether a number is even or odd using modulus.
3. The same logic is implemented in C++ using a function.
4. Input is taken using input() in Python and cin in C++.
5. Output is displayed using print() in Python and cout in C++.
6. Both programs produce identical results for the same inputs.
7. Multiple test cases confirm correctness and consistency.
8. This ensures logical equivalence across both languages.